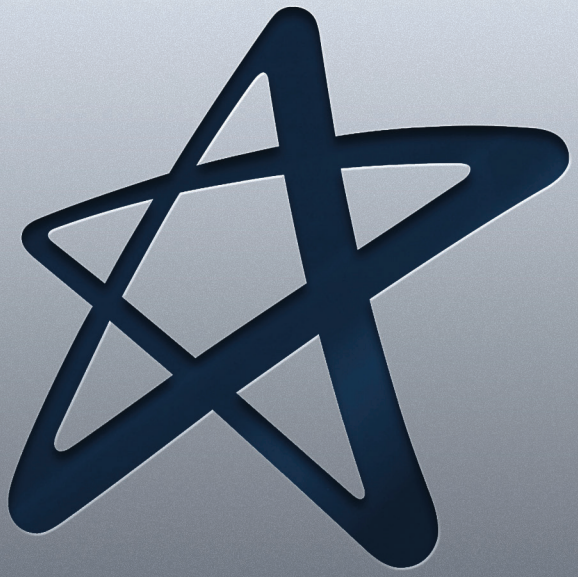
Linguagens Formais e Autômatos



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico

Hierarquia de Classe Linguagens e Tese de *Church*

Responsável pelo Conteúdo:
Prof. Dr. Cleber Silva Ferreira da Luz

Revisão Textual:
Prof.ª Me. Natalia Conti

UNIDADE Hierarquia de Classe Linguagens e Tese de *Church*

- Introdução;
- Tese de *Church*;
- Complexidade;
- Aplicabilidade da Teoria da Linguagem Formal.



OBJETIVO DE APRENDIZADO

- Apresentar os conceitos básicos sobre a Hierarquia de classes linguagens.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu "momento do estudo";
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Nesta unidade iremos estudar a Hierarquia de classe linguagens e a Tese de *Church* e, também, iremos estudar um pouco mais sobre complexidade na teoria da linguagem forma. A Hierarquia de classe linguagens e a Tese de *Church* são apresentados com detalhes na Seção 2. A Complexidade é explorada na Seção 3.

Ao longo da disciplina, foi falado por diversas vezes que a teoria da linguagem formal é muito aplicada em linguagens artificiais, aquelas linguagens que são utilizadas na programação. Para finalizar o nosso estudo com chave de ouro, iremos estudar com detalhes quais elementos e onde estes elementos são utilizados nas linguagens artificiais. Este estudo é realizado na Seção 4.

Tese de *Church*

Na unidade anterior estudamos a **Máquina de *Turing*** que foi proposto por Alan Turing em 1936 (RAMOS, M.; VEJA, I.; JOSE, J.; 2009). Este formalismo matemático explora os limites da capacidade de expressar soluções de problemas. Podemos dizer que esse formalismo apresentado por *Turing* trata-se de uma proposta de definição formal da noção intuitiva de algoritmos.

Vamos lembrar um pouco sobre a máquina de *Turing*. A máquina de *Turing* pode ser considerada como um mecanismo simples que formaliza a ideia de uma pessoa que realiza cálculos. A ideia de uma pessoa realizar cálculos pode ser vista como uma máquina formada de três partes, tais como:

- **Fita:** que é utilizada tanto como, dispositivo de entrada e saída, como dispositivo de memória de trabalho;
- **Unidade de Controle:** que mostra o estado atual da máquina. Este possui uma unidade de leitura e gravação (cabeça da fita), a qual acessa uma célula da fita de cada vez e se movimenta para a esquerda ou para a direita;
- **Programa, Função Programa ou Função de Transição:** Este define o estado da máquina e comanda as leituras, as gravações e o sentido de movimento da cabeça.

Inicialmente, podemos observar que a fita é finita. Ela é dividida em células, cada uma armazena um símbolo. Os símbolos podem:

- Pertencer ao alfabeto de entrada;
- Pertencer ao alfabeto auxiliar;
- Ser “branco”;
- Ser “marcador de início de fita”.

No início do processamento, a palavra a ser processada ocupa as células mais à esquerda após o marcador de início de fita, ficando as demais com "branco", a Figura 1 ilustra esse procedimento. Nesta Figura, o início da fita é representado por χ e o branco é representado por β. Por sua vez, a unidade de controle possui um número finito e predefinido de estados. Nesta unidade a cabeça da fita lê o símbolo de uma célula de cada vez e grava um novo símbolo. Após realizar a leitura ou gravação, a cabeça move uma célula para a direita ou para a esquerda. O sentido do movimento e o símbolo gravado são definidos pelo programa.

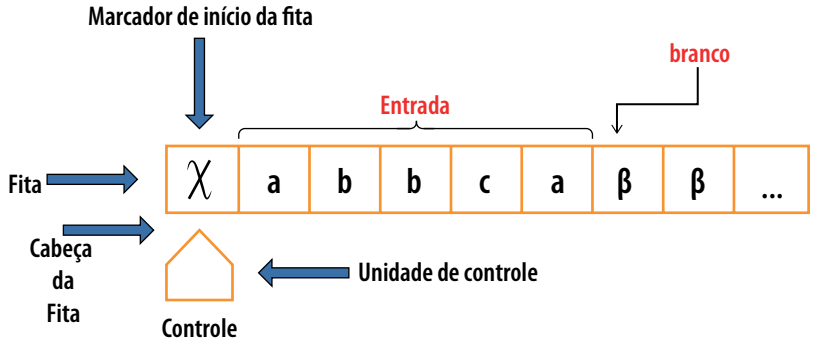


Figura 1 – Máquina de *Turing*: fita e unidade de controle

Na literatura, podemos encontrar diversos trabalhos que resultaram em conceitos semelhantes à "Máquina de *Turing*", tais como a "Máquina de *Post*", proposto por Post em 1936 e "Funções Recursivas" proposta por Kleene também 1936. Pelo fato de trabalhos diferentes e independentes gerarem resultados semelhantes em termos de capacidade de expressar computabilidade só reforça a **Tese de *Church***. Esta tese também conhecida como **hipótese de *Church***, ou ainda, como **hipótese de *Turing-Church*** (MENEZES, P., 2011).

Essa tese diz que "A capacidade de computação representada pela máquina de *Turing* é o limite máximo que pode ser atingido por qualquer dispositivo de computação". Podemos concluir que a tese de *Church* afirma que qualquer outra forma de expressar algoritmos terá, no máximo, a mesma capacidade computacional da máquina de *Turing*.

Hierarquia de classe linguagens

A Hierarquia de Classe de Linguagens é formada pelas linguagens a seguir:

- Regulares que também são conhecidas como linguagens do Tipo 3;
- Livres de Contexto ou do Tipo 2;
- Sensíveis ao Contexto ou do Tipo 1;
- Recursivamente Enumeráveis ou do Tipo 0.

Essa hierarquia de classe de linguagens também é conhecida como hierarquia de *Chomsky*. Noam Chomsky definiu esta classe como modelos para as linguagens naturais. A Figura 2 ilustra a hierarquia de *Chomsky*.

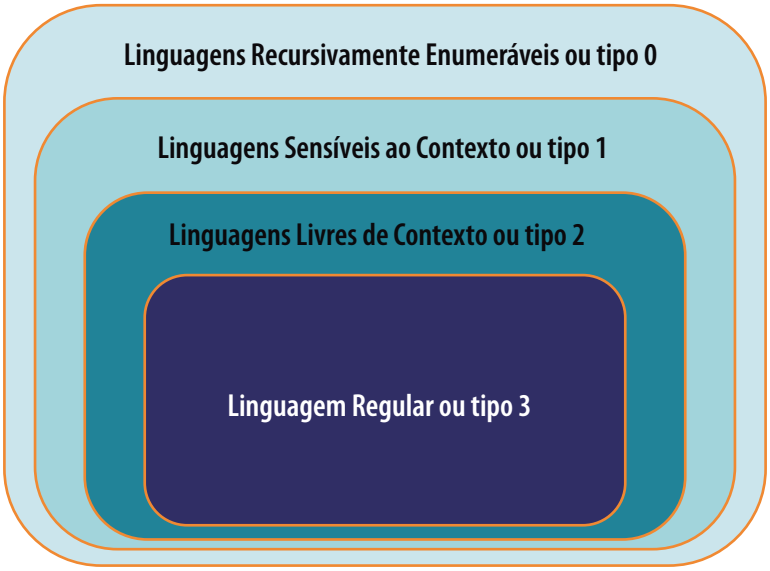


Figura 2 – Hierarquia de *Chomsky*

Em contrapartida, nem sempre as linguagens artificiais, linguagens de programação são tratadas adequadamente na Hierarquia de *Chomsky*. Existem diversas linguagens de programas, entre as quais, encontram-se as linguagens que não são livres do contexto, para estas, o poder dos formalismos sensíveis ao contexto é excessivo, sendo inadequados principalmente no que se refere à complexidade. Além disso, o conhecimento das linguagens sensíveis ao contexto é relativamente limitado, o que dificulta o seu tratamento. A seguir, são apresentados alguns exemplos de problemas que não podem ser tratados na classe de linguagem livres de contexto:

- Múltiplas ocorrências de um mesmo trecho de programa, como por exemplo, a declaração de um identificador e suas referências de uso (assim como o problema análogo ao da linguagem {wcw | w é palavra de {a, b}* }, a qual não é livre de contexto);
- Casos de validações de expressões com variáveis de expressões com variáveis de tipos diferentes;
- Associações semânticas de um trecho de programa, que dependeria da análise de um conjunto de informações como identificadores, ambientes, tipos de dados, localização, sequências de operações, entre outros.

Em contrapartida, para algumas linguagens de programação, a Classe das Linguagens Livres do Contexto é excessiva e a Classe das Linguagens Regulares, insuficiente. Podemos citar como exemplo, o formalismo Autômato com Pilha que possui a facilidade de não-determinismo. Todavia, se a linguagem pode ser denotada por um Autômato com Pilha Determinístico, assim, é possível implementar um reconhecedor com tempo de processamento proporcional a 2^n (n é o tamanho da entrada), o que é muito mais eficiente que o melhor algoritmo conhecido para as linguagens livres do contexto.

Podemos dizer que, tanto para linguagens artificias como para as linguagens naturais, o estudo da Classe das Linguagens Livres do Contexto tem sido de especial

interesse, pois elas permitem uma representação simples da sintaxe, adequada tanto para a estruturação formal, como para a análise computacional. Todavia, o estudo dessa classe tem mostrado problema não-solucionáveis, como por exemplo:

- Determinar se uma gramática livre do contexto é ambígua;
- Não existe um algoritmo que verifique a igualdade de duas linguagens livres de contexto, o que dificulta a otimização e o teste de processadores de linguagens.

Assim, dependendo da linguagem e dos objetivos, alguns estudos específicos, eventualmente fora da Hierarquia de *Chomsky*, são recomendados ou necessários.

Complexidade

Analisar a complexidade de um algoritmo é muito importante. Nesta seção iremos estudar a complexidade ligada à linguagem formal. Todavia, porque estudar complexidade é importante? Porque estudar a complexidade de um algoritmo? Essas perguntas são respondidas a seguir. Devemos observar e entender a complexidade de um algoritmo porque:

- A preocupação com a complexidade de algoritmos é fundamental sabe se o algoritmo é eficiente ou não;
- É importante também para comparar dois algoritmos e saber qual é o mais eficiente entre eles.

As classes de complexidade visam classificar problemas computacionais de acordo com sua dificuldade, e relacionar essas classes entre si. Aqui, iremos estudar as classes de complexidade P, NP e NP-completa.

Classe P

Na classe P encontra-se o conjunto de problemas que são resolvidos em tempo polinomial por uma máquina de *Turing* determinística. A acrônica P em inglês significa *Polynomial*. Nesta classe consistem os problemas que podem ser resolvidos em tempo de execução $O(n^k)$, sendo k uma constante.

Classe NP

A classe NP possui o conjunto de problemas que são solucionados em tempo polinomial por uma máquina de *Turing* não-determinística. A acrônica NP significa em inglês Tempo polinomial não determinístico (*Non-Deterministic Polynomial time*).

Em palavras mais simples, podemos definir NP como sendo uma classe que possui o conjunto de problemas que podem possuir uma solução obtida em tempo polinomial por uma máquina de *Turing* não determinística.

Classe NP-Completa

Nesta classe de complexidade encontra-se o subconjunto dos problemas NP de tal modo que todo problema em NP pode-se reduzir, com uma redução de tempo polinomial, a um dos problemas NP-completo. Dessa forma, os problemas de NP-completo são os problemas mais difíceis de NP. Caso seja possível encontrar uma maneira de resolver qualquer problema NP-completo rapidamente, então poderiam ser utilizados algoritmos para resolver todos os problemas NP rapidamente.

Saber que um problema é NP-completo pode ser muito útil, pois assim, pode-se evitar que se desperdice tempo tentando encontrar um algoritmo de tempo polinomial para resolver um problema, quando esse algoritmo não existe.

Aplicabilidade da Teoria da Linguagem Formal

A teoria das linguagens formais surgiu em 1950. Inicialmente, essa teoria visava apenas o estudo das teorias relacionadas com as linguagens naturais. Posteriormente, a teoria das linguagens formais foi aplicada no estudo das linguagens artificias, como as originárias da computação, principalmente nas linguagens de programação.

É possível dizer que as linguagens formais possuem um papel de suma importância nas linguagens de programação. Nas linguagens de programação são usados os mecanismos de reconhecimento e geração de linguagens da teoria da linguagem formal. Podemos observar, ainda, que nas linguagens de programação a teoria das linguagens formais possui papel fundamental nas **análises léxica** e **sintática** das linguagens. Análise léxica e sintática são etapas do processo de compilação de uma linguagem de programação. A próxima seção apresenta todos os detalhes sobre o processo de compilação e, também, detalha quais são os mecanismos da teoria da linguagem formal que são usados neste processo.

Processo de tradução das linguagens de programação

Habitualmente, as linguagens de programação são “Compiladas” ou “Interpretadas”. Todavia, o que significa uma linguagem ser “Compilada” ou “Interpretada”? Vamos entender melhor esses conceitos agora. Com certeza, você já deve ter ouvido que os **computadores entendem somente zeros e uns**, ou seja, a linguagem que o computador utiliza é a forma **linguagem binária**, que é composta somente por zeros e uns. Quando você cria um programa, você precisa escrever as instruções que o programa irá executar em um arquivo chamado **código fonte**. Este código é escrito em uma linguagem de alto nível, ou seja, uma linguagem que qualquer ser humano consegue entender (um ser humano com conhecimentos básico

em programação). Todavia, o computador entende somente zeros e uns, logo, ele não irá entender o que você escreveu no seu código fonte. E agora? Para resolver esse impasse é necessário fazer uma **tradução**. **Traduzir o que foi escrito em linguagem de alto nível para a linguagem de máquina**. A Figura 3 ilustra o processo de Tradução.



Figura 3 – Processo de Tradução

O Interpretador e o Compilado fazem essa tradução. Podemos dizer que o objetivo dos dois é o mesmo, fazer a tradução de uma linguagem alto nível para linguagem baixo nível. Entretanto, a diferença primordial entre os dois é que o interpretado realiza a tradução em tempo de processamento, isto é, ele toma as instruções do código fonte e as entradas, traduz e executa as instruções logo em seguida. Ao compasso que o compilado executa a tradução e gera um executado, que será executado posteriormente. A Figura 4 ilustra o processo de tradução do interpretador, ao passo que a Figura 5 ilustra o processo de tradução realizado pelo compilador.

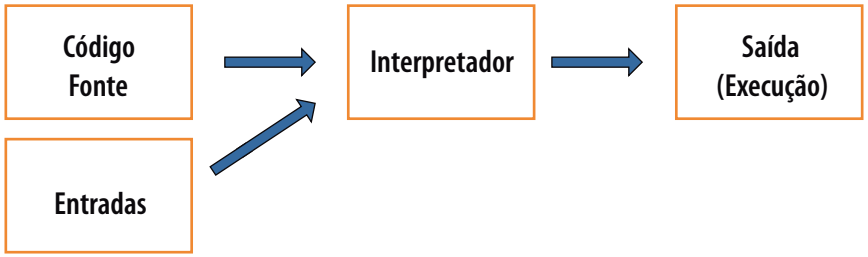


Figura 4 – Processo de tradução realizado pelo interpretador

Interpretador: Um interpretador é outro tipo comum de processador de linguagem. Em vez de produzir um programa objeto como resultado da tradução, um interpretador executa diretamente as operações especificadas no programa fonte sobre as entradas fornecidas pelo usuário.

Compilador: Um compilador é um programa que recebe como entrada um programa em linguagem de programação – a linguagem fonte – e o traduz para um programa.



Figura 5 – Processo de tradução realizado pelo compilador

Como exemplo de linguagens de programação **interpretadas**, podemos citar ***Python*** e **R**. Já sobre as **linguagens compiladas** podemos citar como exemplo as linguagens **C**, **C++** e ***Java***.

Processo de Compilação

O processo de tradução que o interpretador realiza é muito simples, ao passo que o processo de tradução do compilador é complexo e com muitas etapas. No processo de compilação, o uso de alguns mecanismos da linguagem formal é extremamente visível. Por isso, a partir de agora, vamos focar mais no processo de tradução realizado pelo compilador.

O processo de compilação é composto por duas fases, que são:

- "Análise";
- "Síntese".

Na fase de **análise** são realizadas diversas análises no código do usuário. Análises que são necessárias para verificar se o que usuário escreveu condiz com a linguagem. Se é uma tradução, então, a tradução deve estar, coerente. Já na fase de **síntese**, é gerado o código binário, uma vez que a tradução "bateu", então é gerado o código binário da linguagem. A fase de síntese também é chamada de **geração de código**.

A fase de análise é composta por outras 3 sub etapas, que são as etapas de:

- Análises léxicas;
- Sintática;
- Semântica.

Ao passo que a fase de geração de código (síntese) é composta pelas etapas de:

- Geração de código intermediário;
- Otimização;
- Geração de código.

A Figura 6 ilustra todo o processo de compilação de uma linguagem.

A teoria da linguagem formal é aplicada na fase de análise do compilador. Nas análises léxicas, sintáticas e semânticas. A próxima seção apresenta detalhes sobre a análise léxica.



Figura 6 – Processo de Compilação

Análise léxica

A **Análise léxica** é realizada por um analisador léxico. O analisador léxico é responsável por varrer o código fonte e encontrar padrões. Vamos considerar a linguagem C para exemplificar. A declaração de variáveis e atribuição na linguagem C é realizada da seguinte forma:

```
int x = 2;
```

Primeiro o tipo de dado, depois o nome da variável e por último, a atribuição.

Como falado anteriormente, o analisador léxico percorre o código fonte que o usuário escreveu para reconhecer padrões. Por exemplo, os caracteres **i**, **n**, **t** formam um padrão, que é a palavra **reservada int** e, que por sinal, é uma palavra aceita na linguagem C. Agora, por exemplo, vamos supor que o usuário escreveu a seguinte instrução:

´int x = 2;

O usuário escreveu o caractere ´ (agudo) que não é aceito na linguagem C. Dessa forma, o agudo seguido de qualquer caractere não casa com nenhum padrão especificado na linguagem. Assim, é gerado um erro léxico. Podemos afirmar, também, que o analisador léxico percorre o código fonte para verificar se todos os caracteres presentes no código fonte pertencem à linguagem. No exemplo acima o acento agudo não pertence a linguagem C. Assim, é gerado um erro léxico.

Na análise léxica é percorrido o código fonte para encontrar padrões, certo? Na teoria da linguagem formal, quais foram os mecanismos que estudamos que são capazes de reconhecer padrões? Se você se lembrou das expressões regulares e autômatos acertou!

Análise Léxica: é verificado se os caracteres do programa fonte pertencem ao alfabeto da linguagem. Caso o caractere não pertença ao alfabeto da linguagem, é gerado um erro léxico.

Vimos que as expressões regulares e autômatos são reconhecedores de linguagem. Dessa forma, eles também são capazes de reconhecer padrões. Estudamos, também, que as expressões regulares e os autômatos possuem o mesmo poder computacional. Todavia, habitualmente, os analisadores léxicos usam mais autômatos para reconhecer padrões do que as expressões regulares. Os autômatos oferecem a facilidade do "não determinístico", lembra? Assim, o autômato torna-se mais atraente para os analisadores léxicos. Se você não lembra o que é um autômato, ou muito menos se lembra o que significa um autômato não determinístico, vale a pena conferir novamente em unidades anteriores. Todavia, o quadrado a seguir relembra, rapidamente, o que é um autômato.

Autômato: é um formalismo matemático reconhecedor de linguagens. Sendo composto por estados e transações, um Autômato reconhece se uma palavra pertence a uma linguagem. Um autômato pode ser: **Determinístico**, isto é, dependendo do símbolo lido e do estado corrente (atual), o sistema pode assumir um único estado bem definido; ou **Não determinístico**, isto é, dependendo do símbolo lido e do estado corrente (atual), o sistema pode assumir um conjunto de estados alternativos. Um autômato ainda pode ser classificado como "**Com movimento vazio**", isto é, dependendo do estado atual e sem ler nenhum símbolo, o sistema pode assumir um conjunto de estados. Para mais detalhes sobre autômatos, ver a unidade anterior.

Análise sintática

A análise sintática é realizada por um analisador sintático. Nesta análise é verificada a estrutura das sentenças, isto é, se os elementos da sentença estão na ordem certa. Por exemplo, vamos supor que o usuário digite a seguinte instrução no código fonte:

int x = 2;

O analisador sintático iria verificar se a ordem dos elementos desta sentença está correta. Agora por exemplo, se o usuário digitasse,

x int 2 =;

seria gerado um erro sintático, porque a ordem dos elementos não está correta. Perceba, os elementos existem na linguagem, por isso, iria passar pelo analisador léxico. Agora, não iria passar pelo analisador sintático, pois a ordem dos elementos não está correta, perante a linguagem. A análise sintática transforma uma sentença em uma estrutura de dados, em geral uma árvore, que facilita a captura hierárquica implícita desta sentença.

Em unidades anteriores estudamos a Gramática Livre de Contexto, lembra? A gramática livre de contexto permite gerar uma árvore sintática. Assim, a gramática livre de contexto é usada nos analisados sintáticos para analisar a estrutura (ordem) da sentença. Por exemplo, a exemplo 3 * 5 + 4 poderia ser expressa pela árvore apresentada na Figura 7.

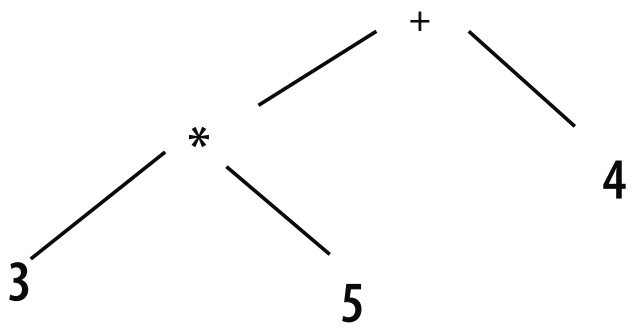


Figura 7 – Árvore para a expressão 3 * 5 + 4

Com a árvore de derivação fornecida pela gramática livre de contexto é possível verificar a estrutura da sentença, isto é, é possível verificar a ordem dos elementos da sentença. Por isso, os analisadores sintáticos utilizam a gramática livre de contexto. Se você não lembra o que é uma gramática livre de contexto, ou muito menos árvore de derivação, vale a pena você reler em unidades anteriores novamente. Todavia, a seguir, é apresenta um breve lembrete sobre gramática livre de contexto.

Gramática Livre do Contexto: pode ser considerada como um formalismo gerador. Uma gramática com restrições na forma das regras de produções, tais restrições são definidas de maneira mais livre que as gramáticas regulares. As gramáticas livres de contextos tem a principal forma:

$$A \rightarrow \alpha.$$

Caso ocorra uma derivação de A, a variável A que é o contexto, não depende de qualquer símbolo que a **anteceda**, ou a **suceda**, ou seja, é **livre de contexto**.

O objetivo do analisador sintático é verificar se uma determinada gramática, com uma sequência de símbolos terminais (Frase), é uma frase válida da linguagem.

Análise semântico

No processo de compilação, a última análise realizada é a análise semântica. Nesta análise é verificado se as sentenças que o usuário escreveu no código fonte, realmente fazem sentido. Por exemplo, na linguagem *JAVA*, a seguinte instrução:

int x = "nome";

iria gerar um erro semântico. Todavia, note que, esta instrução não iria gerar um erro léxico, pois dois dos elementos pertencem à linguagem *JAVA*, isto é, são aceitos pela linguagem. Também, não iria gerar um erro sintático, pois todos os elementos estão na ordem certa. Todavia, iria gerar um erro semântico. Porque estou querendo colocar um dado do tipo "*String*" palavra e uma variável do tipo inteiro. Essa atribuição não faz sentido, uma vez que segundo a linguagem a variável somente pode aceitar dados do mesmo tipo.

Na análise semântica não existe uma "receita" de como realizar a análise semântica. Uma vez que a análise semântica é muito atrelada com a linguagem, isto é, a análise semântica vai refletir o que a linguagem aceita ou não, e isso varia muito de linguagem para linguagem. Por exemplo, na linguagem R a atribuição de variável não é atrelada com o seu tipo, assim, declarações como x = "nome" ou x = 2.57 estão corretas. Todavia, para realizar a análise semântica, muitas linguagens utilizam a árvore de derivação fornecida pelo analisador sintático. Na árvore de derivação, a cada elemento da árvore é atrelado o seu tipo de dado. Assim, basta apenas percorrer a árvore e verificar se o tipo de dado está coerente com o seu elemento.

Para finalizar, vamos ressaltar que o analisador semântico também utiliza o mecanismo da linguagem formal "árvore de derivação".

Geração de Código

Após realizar todas as análises, o próximo passo é gerar o executável. Nas etapas de geração de código não são utilizados nenhum mecanismo da teoria da linguagem formal. Por isso, elas não serão detalhadas aqui. Todavia, se você se interessou pelo assunto, você pode obter mais detalhes em (Price, 2001).

Em Síntese

Podemos observar que as linguagens formais oferecem meios para modelar e desenvolver ferramentas que especificam linguagens e seus processos de análises, bem como suas propriedades e limitações algorítmicas.

Podemos definir linguagem formal como mecanismos formais para representação e especificação de linguagens. Habitualmente, as representações são realizadas por reconhecedores e geradores. Os reconhecedores são mecanismos formais que são utilizados para verificar se uma palavra pertence ou não pertence a uma linguagem. Os Geradores são mecanismos formais que permitem a geração de palavras de uma linguagem.

Nesta unidade são estudadas as classes de complexidade no contexto de linguagens. Analisar a complexidade de um algoritmo é muito importante. As classes de complexidade visam classificar problemas computacionais de acordo com sua dificuldade, e relacionar essas classes entre si. Aqui, iremos estudar as classes de complexidade P, NP e NP-completa. Na **classe P** encontra-se o conjunto de problemas que são resolvidos em tempo polinomial por uma por uma máquina de Turing determinística. A acrônica P em inglês significa *Polynomial*. A **classe NP** possui o conjunto de problemas que são solucionados em tempo polinomial por uma máquina de Turing não-determinística. A **classe NP-Completa** é o subconjunto dos problemas NP de tal modo que todo problema em NP pode-se reduzir, com uma redução de tempo polinomial, a um dos problemas NP-completo.

Nesta unidade estudamos também a aplicabilidade da teoria da linguagem formal. Vimos que o processo de compilação utiliza vários recursos da teoria da linguagem formal. O processo de compilação é composto por duas fases, que são: "análise" e "síntese".

Na fase de **análise** são realizadas diversas análises no código do usuário. Nesta fase são executadas 3 sub etapas, que são as etapas de análises léxicas, sintática e semântica.

Na fase de análise léxica, o código fonte é varrido para encontrar padrões. O mecanismo da teoria da linguagem formal utilizado nesta fase são os autômatos. Na fase sintática é verificado se os elementos da linguagem estão na ordem correta. Assim, o mecanismo utilizado nesta fase é a Gramática Livre de Contexto. Na fase semântica é verificado se a sentença realmente faz sentido e, também, é utilizado a árvore de derivação.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Introdução à Teoria de Autômatos, Linguagens e Computação

HOPCROFT, J. E., ULLMAN, J. D. **Introdução à Teoria de Autômatos, Linguagens e Computação**. Campus, 2002.

Theory of Computation

SIPSER, M. ***Theory of Computation***, India, 2008.

Introdução à teoria da computação

SIPSER, M. **Introdução à teoria da computação**, Cengage, 2005.

Linguagens Formais e Autômatos: 3

MENEZES, P. B. **Linguagens Formais e Autômatos:** 3. Bookman, 2010.

Implementacao de Linguagens de Programacao: Compiladores

PRICE, A. M. A. **Implementacao de Linguagens de Programacao:** Compiladores. 2. ed. Porto Alegre: Sagra Luzzatto, 2001.

Referências

GERSTING, J. **Fundamentos matemáticos para a ciência da computação**. 5. ed. Rio de Janeiro: LTC. 2004. 538p.

HOPCROFT, J. E.; ULLMAN, J. D.; MOTWANI, R. **Introdução à teoria de autômatos, linguagens e computação**. Rio de Janeiro: Campus, 2001.

HOPCROFT, J. E. ***Introduction to automata theory, languages and computation***. Massachusetts: Addison-wesley Publishing Cia., 1979. 418 p.

MENEZES, P. B. **Linguagens formais e autômatos**. 6. ed. São Paulo: Bookman, 2011.

PAPADIMITRIOU, C. H.; LEWIS, H. R. **Elementos da Teoria da Computação**. 2. ed. Porto Alegre: Bookman. 2000. 344p.

RAMOS, M. V. M.; VEGA, I. S.; JOSE NETO, J. **Linguagens formais:** teoria, modelagem e implementação. Porto Alegre: Bookman, 2009. 656 p.

SIPSER, M. **Introdução à teoria da computação**. 2. ed. São Paulo: Thompson Learning, 2007.

VIEIRA, N. J. **Introdução aos fundamentos da computação:** linguagens e máquinas. São Paulo: Thomson, 2006. 319 p.


Cruzeiro do Sul
Educacional